

LABORATORY OBSERVATION

Course: Full Stack Web Development

Course Code: 23CM4121

Experiment No.: 10

Student Name: Gorapalli Sravya

Roll No.: A24126552257

Date: 26-08-2026

1. TITLE

Develop an Interactive To-Do List Using DOM and Event Listeners

2. AIM

To develop an interactive to-do list web application using DOM manipulation and JavaScript event listeners, allowing tasks to be added, marked complete and deleted.

3. OBJECTIVE

The objectives of this experiment are:

- To understand and use the `addEventListener()` method to handle click and keyup events.
- To dynamically add tasks to a list using the DOM.
- To mark a task as complete by toggling a CSS class.
- To delete a task from the list using event delegation.

4. THEORY

The `addEventListener()` method attaches an event handler to an element without overwriting any existing handlers on that element, and supports common events such as click and keyup.

Event delegation is a technique where a single event listener is attached to a parent element (such as a `<ul>`) to handle events triggered by its child elements. This is especially useful for elements such as list items that are created dynamically at runtime, since a listener does not need to be attached individually to each new item.

The event object's `target` property identifies the exact element that triggered the event, which allows a delegated listener to determine which button (Complete or Delete) was actually clicked and act on the corresponding list item.

5. ALGORITHM / PROCEDURE

1. Create an HTML file with a text input, an Add button and an empty unordered list.
2. Select the input, button and list elements using the DOM.
3. Define an `addTask()` function that creates a new `<li>` containing the task text, a Complete button and a Delete button.

4. Attach `addTask()` to the Add button's click event using `addEventListener()`.
5. Attach a keyup listener to the input so pressing Enter also adds the task.
6. Attach a single click listener to the `<ul>` (event delegation) to handle Complete/Delete clicks on any current or future item.
7. On a Complete click, toggle a 'completed' CSS class on the parent `<li>` to show a strikethrough.
8. On a Delete click, remove the parent `<li>` from the DOM.
9. Save the file and test adding, completing and deleting tasks in the browser.

6. PROGRAM

index.html

```
<!DOCTYPE html>
<html>
<head>
  <title>Interactive To-Do List</title>
</head>
<body>
  <h1>My To-Do List</h1>
  <input type="text" id="taskInput" placeholder="Enter a task and press Enter">
  <button id="addBtn">Add</button>

  <ul id="taskList"></ul>

  <script src="script.js"></script>
</body>
</html>
```

script.js

```
const taskInput = document.getElementById("taskInput");
const addBtn = document.getElementById("addBtn");
const taskList = document.getElementById("taskList");

function addTask() {
  const text = taskInput.value.trim();
  if (text === "") return;

  const li = document.createElement("li");

  const span = document.createElement("span");
  span.textContent = text;

  const completeBtn = document.createElement("button");
  completeBtn.textContent = "Complete";
  completeBtn.className = "complete-btn";

  const deleteBtn = document.createElement("button");
  deleteBtn.textContent = "Delete";
  deleteBtn.className = "delete-btn";

  li.appendChild(span);
  li.appendChild(completeBtn);
  li.appendChild(deleteBtn);
  taskList.appendChild(li);
}
```

```

    taskInput.value = "";
  }

  addBtn.addEventListener("click", addTask);

  taskInput.addEventListener("keyup", (e) => {
    if (e.key === "Enter") {
      addTask();
    }
  });

  // Event delegation for Complete / Delete buttons
  taskList.addEventListener("click", (e) => {
    if (e.target.classList.contains("complete-btn")) {
      e.target.parentElement.classList.toggle("completed");
    }
    if (e.target.classList.contains("delete-btn")) {
      e.target.parentElement.remove();
    }
  });

```

7. CODE EXPLANATION

Tag / Code	Explanation
<code>addEventListener("click", ...)</code>	Attaches a click event handler to the Add button
<code>addEventListener("keyup", ...)</code>	Detects when the Enter key is released inside the input field
<code>document.createElement()</code>	Dynamically creates the list item, span and buttons for a task
<code>taskList.appendChild()</code>	Adds the newly created task item to the visible list
<code>e.target</code>	Identifies the exact button that was clicked inside the list
<code>classList.toggle("completed")</code>	Adds or removes the CSS class that shows the strikethrough style
<code>element.remove()</code>	Removes a task's <code></code> element from the DOM
Event delegation on <code></code>	A single listener on the list handles clicks for all current and future tasks

8. TEST CASES AND EXECUTION DATA

The to-do list was tested in the browser by adding, completing and deleting tasks.

Test Case	Description	Expected Result	Actual Result	Status
TC-01	Type a task and click Add	Task appears at the bottom of the list	Task appeared correctly	Pass
TC-02	Type a task and press Enter	Task is added without clicking Add	Task added via Enter key	Pass
TC-03	Click Complete on a task	Task text shows a strikethrough style	Strikethrough applied correctly	Pass
TC-04	Click Delete on a task	Task is removed from the list	Task removed correctly	Pass

TC-05	Click Add with an empty input	No empty task is added	No empty task added	Pass
-------	-------------------------------	------------------------	---------------------	------

9. OUTPUT

Output: Tasks Added, Completed and Deleted

My To-Do List

Complete lab record submission

Revise JavaScript event listeners

Prepare for viva

Three tasks added to the list (one via Add button, two via Enter key)

My To-Do List

Complete lab record submission

Complete

Delete

Revise JavaScript event listeners

Complete

Delete

After marking the first task complete and deleting the third task

10. RESULT

Thus, an interactive to-do list was successfully developed using DOM manipulation and JavaScript event listeners, and task addition, completion and deletion were verified in the browser.